

KG-Commit: A Knowledge Graph for Commit-Level Bug Detection

Design, Schema, Online Growth, and the V1→V2 Evolution

apache/groovy | ApacheJIT-labelled subset

June 30, 2026

Abstract

This document describes the first complete version of **KG-Commit**, a Neo4j knowledge graph built to support *commit-level (just-in-time) bug detection* on the **apache/groovy** project. The graph fuses three layers: (i) a *structural* layer of commit history (commits, developers, files, issues); (ii) an *Abstract Syntax Tree (AST)* layer that stores the actual code structure of every source file; and (iii) a *delta* layer that connects each commit directly to the precise AST nodes it changed. The graph grows *online*: commits are ingested in chronological order, and as each new commit arrives its modifications are modelled as a small “delta graph” laid on top of an evolving AST, so the whole history of every file is captured without storing a full AST snapshot per commit. Each commit carries a **buggy/benign** label from the ApacheJIT dataset, making the graph directly usable for training and online evaluation of defect-prediction models. We document every node and edge type, the exact mechanics of what happens when a commit arrives, the correctness guarantees, the strengths and limitations of the design for our task, and the planned next steps. We also report a first, deliberately GPU-free inference suite: across nine classical and graph-native methods evaluated online (prequential), an early fusion of the KG’s structural/relational signals with the classic JIT metrics lifts PR-AUC from 0.398 to 0.616 (+55% relative) over the metrics-only baseline — evidence that the AST/delta layers carry real predictive value for commit-level bug detection without any GNN. This document also records the **V2** cycle that finalizes the graph: an in-place data-quality cleanup, recovery of every previously-unparseable `.java` file (javalang + tree-sitter), a commit-message text feature, a baselines-vs-V1-vs-V2 comparison on the online operating-point metric, and a transparent pre-submission review of the build/inference code. We close with a complete inventory of the issues that remain (carried over from V1 or newly found in V2) and a two-track roadmap: per-issue remediation, and a KG refinement plan whose centerpiece is promoting commit text from a node feature to a first-class *textual/semantic graph layer*.

Contents

1	Goal and Scope	2
2	High-Level Architecture	3
3	Node Types	3
4	Edge Types	4
5	The Base Graph	5
6	Online Growth: What Happens When a Commit Arrives	6
6.1	Diffing a modified file	6
6.2	Applying the delta and evolving the structure	7

6.3	Node identity across versions	7
6.4	Why this is space- and time-efficient	7
7	Labels for Bug Detection	7
8	Statistics (finalized full build)	8
9	Audit and Data Quality of the Finalized KG	9
10	Strengths and Limitations for Commit-Level Bug Detection	9
11	Downstream Inference	10
11.1	Problem framing (theory)	10
11.2	Features from the KG (theory + implementation)	11
11.3	Methods (theory + implementation)	11
11.4	Evaluation protocols	11
11.5	Results	11
11.6	Interpretation	12
11.7	Controlled comparison to the project baselines	13
11.8	Fully-online (prequential) evaluation and operating-point F1	14
12	Version 2 — Debugging and Refinement	17
12.1	Data-quality debugging	17
12.2	Commit-message text feature	17
12.3	Statistical comparison: V1 → V2	19
12.4	Performance comparison: V1 → V2	19
12.5	Baselines vs. KG-v1 vs. KG-v2 on online buggy-F1	20
12.6	V2 finalization: known issues and their status	21
13	V2 Closeout: Outstanding Issues	22
13.1	Carried over from V1 (not fixed in V2)	22
13.2	Newly detected in V2 (finalization review)	23
14	Roadmap A — Remediation of the Outstanding Issues	24
15	Roadmap B — KG Refinement and the Path to V3	25
15.1	B1. Commit text as a first-class graph layer (V3 centerpiece)	25
15.2	B2. A heterogeneous GNN over the multimodal graph	26
15.3	B3. Richer code semantics beyond the AST	26
15.4	B4. Bug-localisation supervision	26
15.5	B5. Temporal / dynamic-graph modelling	26
15.6	B6. Evaluation, calibration, and generalisation	26
16	Legacy Layer	27
A	Example Cypher	27

1 Goal and Scope

The objective is a graph from which a model can *infer whether a commit is likely to introduce a bug*, using both the change itself and the surrounding code structure. Concretely:

- **Project:** apache/groovy (a full local git clone).

- **Commit scope:** the **8,059 commits** that appear in the **ApacheJIT** dataset for groovy (`data/apachejit/projects/apache_groovy.csv`), spanning 2003–2019. These are the only commits we treat as the labelled study set; the local database also contains the surrounding history as **Commit** nodes, but only the ApacheJIT commits are flagged `in_jit=true` and grown into the AST/delta layers.
- **Base snapshot:** commit `408b29851d7bbe4d343340832297e4be7e0c5578` (“BASE”), the first substantial Java commit, whose 120 `.java` files seed the AST layer.
- **Label:** `buggy` $\in \{\text{True}, \text{False}\}$ from ApacheJIT (1,614 buggy / 6,445 benign $\approx 20\%$ positive).

2 High-Level Architecture

The graph has three conceptual layers that share the same node space:

1. **Structural / process layer** — the classical commit graph: **Project**, **Branch**, **Developer**, **Commit**, **File**, **Issue**, and the relationships between them (authorship, parenthood, file changes, issue fixes).
2. **AST layer** — for every tracked source file, a tree of **ASTNodes** linked by `AST_CHILD`, rooted at the **File** via `HAS_AST`. Seeded from the 120 BASE files and extended as new files appear.
3. **Delta layer** — four relationship types (`ADDS`, `REMOVES`, `UPDATES`, `MOVES`) that connect a **Commit** *directly* to the AST nodes it changed. This is the heart of the commit-level model.

A small *legacy* layer (**ASTDiff**/**ASTEdit**) from an earlier experiment is still present but **deprecated** in favour of the delta layer (§16).

Theory. The design is a *heterogeneous information network* (HIN): typed nodes and typed edges in one graph. This lets a single model reason jointly over *process* evidence (who changed what, when, how often) and *code-structure* evidence (the syntactic shape of the change and its surroundings) — the two signal sources the defect-prediction literature treats separately. Anchoring commit changes to *shared*, *persistent* AST nodes (rather than to per-commit summary blobs) is what enables relational and neighbourhood reasoning across the whole history.

Practice. The structural/process layer is produced in `kg_commit/knowledge`, (driven from the ApacheJIT CSV and the local git clone). The AST layer is seeded once (`build_file_ast_graphs.py` + `ingest_base_kg_and_ast.py`), positions are stamped (`add_positions_to_astnodes.py`), labels are applied (`apply_commit_labels.py`), and the delta layer is grown by the online engine (`build_online_kg.py`, using `online_ast_diff.py`). Everything is stored in one Neo4j database.

3 Node Types

Table 1 lists every node label, its meaning, and its key properties.

Table 1: Node types.

Label	Meaning	Key properties
Project	The repository.	<code>id</code> (e.g. <code>apache/groovy</code>).
Branch	A git branch.	<code>id</code> .
Developer	A commit author.	<code>id</code> (name/email).

Label	Meaning	Key properties
Commit	A single commit.	id (40-char SHA), message, committed_at, author_ts (UNIX seconds = online order), in_jit, buggy, is_fix, jit_year, and the ApacheJIT change metrics la, ld, nf, nd, ns, ent, ndev, age, nuc, aexp, arexp, asexp.
File	A repository-relative file path.	id (path).
Issue	A referenced issue (e.g. GROOVY-20).	id.
ASTNode	One node of a file's AST (declaration, statement, expression, literal, or identifier leaf).	id = {file}::A{i} for original nodes or {file}::D{sha8}:{i} for nodes inserted by a commit; file, ast_type (javalang type), group (declaration/statement/expression/literal/leaf), value, is_leaf, depth, pos_line, pos_col, and on evolving nodes alive (false once removed), removed_by, is_delta (true if inserted by a commit).
ASTDiff (<i>legacy</i>)	Per-(commit,file) diff summary.	id, commit_id, file, n_insert/delete/update/move, total.
ASTEdit (<i>legacy</i>)	A single GumTree edit action.	action, node_type, node_label, new_label, parent_type, position.

AST node identity. Node ids are *deterministic*: the same source file always yields the same {file}::A{i} ids, where *i* is a pre-order DFS counter from the AST builder. This determinism is what lets us thread node identity across versions of a file (§6).

4 Edge Types

Table 2 groups the relationships by layer.

Table 2: Relationship types.

Type	From → To	Meaning / properties
<i>Structural / process layer</i>		
BELONGS_TO	Commit → Project	Commit is part of the project.
INSIDE_BRANCH	Branch → Commit	Commit is on the branch.
PARENT_OF	Commit → Commit	Git parent relation (history spine).
AUTHORED_BY	Commit → Developer	Authorship.
FIXES_ISSUE	Commit → Issue	Commit message references/fixes an issue.
<i>File-change layer</i>		

Type	From → To	Meaning / properties
ADDED	Commit → File	File added by the commit.
MODIFIED	Commit → File	File modified by the commit.
DELETED	Commit → File	File deleted by the commit.
RENAMED_FROM	Commit → File	File rename endpoints.
and RE-NAMED_TO		
<i>AST layer</i>		
HAS_AST	File → ASTNode	Links a file to its AST root node.
AST_CHILD	ASTNode → ASTNode	Parent-to-child tree edge; property <code>pos</code> = child ordinal.
<i>Delta layer (the commit-level model)</i>		
ADDS	Commit → ASTNode	The commit <i>inserted</i> this node (node has <code>is_delta=true</code>); the node is also wired into the tree via a new <code>AST_CHILD</code> edge to its parent.
REMOVES	Commit → ASTNode	The commit <i>deleted</i> this node; the node is kept but marked <code>alive=false</code> , <code>removed_by=commit</code> (history preserved).
UPDATES	Commit → ASTNode	The commit changed the node’s value; properties <code>old_value</code> , <code>new_value</code> .
MOVES	Commit → ASTNode	The commit relocated the node; property <code>new_parent_type</code> .
<i>Legacy layer (deprecated)</i>		
HAS_AST_DIFF	Commit → ASTDiff	old per-file diff summary.
EDIT	ASTDiff → ASTEdit	one edit of the diff.
DIFFED_AT	File → ASTDiff	file the diff applies to.

5 The Base Graph

The AST layer is seeded at the BASE commit. Each of the 120 `.java` files is parsed with the `javalang` library and converted to a graph by a pre-order traversal that assigns every node a stable id `{file}::A{i}`. Two kinds of nodes are produced:

- **Structured nodes** ($\approx 36\%$): real `javalang.tree` nodes (classes, methods, fields, statements, expressions, literals), carrying a source position (`pos_line`, `pos_col`).
- **Identifier leaves** ($\approx 64\%$): raw string children (identifier and operator tokens) that `javalang` attaches to a node; these have no source position (`pos_line=-1`).

Each file is attached as `FILE` $\xrightarrow{\text{HAS_AST}}$ `ROOT` $\xrightarrow{\text{AST_CHILD}^*}$ `DESCENDANTS`. At BASE this is 120 files and $\approx 46\text{k}$ AST nodes. This is the “previous structure” onto which all later commit deltas are laid.

Theory. An AST abstracts away surface formatting and records the grammatical structure of code; making it an explicit subgraph (rather than a flat token sequence) lets downstream models exploit parent/child and type-of-node relations. Choosing one consistent parser (`javalang`) for the whole graph is what makes node identities comparable across file versions (§6.3).

Implementation. `build_file_ast_graphs.py` parses each BASE file in a killable subprocess (javalang can hang on pathological input) and emits a `networkx` graph whose node ids are the deterministic `{file}::A{i}` counters; `ingest_base_kg_and_ast.py` MERGEs these into Neo4j under each `File`; `add_positions_to_astnodes.py` re-runs the identical traversal to stamp `pos_line/pos_col` on every node.

6 Online Growth: What Happens When a Commit Arrives

Theory. The problem of describing how one tree becomes another is *tree differencing* / tree edit distance: find a minimal-cost set of insert/delete/update/move operations mapping a “before” tree to an “after” tree. Equivalently, one seeks a *matching* between the two node sets; unmatched before-nodes are deletions, unmatched after-nodes are insertions, and matched nodes whose label differs are updates. We compute this matching and record each operation as a typed edge from the commit to the affected AST node. Modelling change as a delta (operations) rather than as two stored snapshots is both information-preserving and asymptotically smaller: storage grows with the *amount of change*, not with history length $\times$ file size.

Practice. Commits are processed in **chronological order** (`author_ts`). The engine (`build_online_kg.py`) maintains, per tracked file, the *current* state of its AST in the graph. When commit *C* arrives, it computes the set of changed `.java` files (`git diff` of *C* against its first parent) and, for each file, takes one of four actions.

Listing 1: Per-commit processing.

```
for each commit C in stream order (by author_ts):
  for each changed .java file F of C (status A / M / D):

    if status == DELETED:
      mark every alive ASTNode of F as alive=false, removed_by=C
      add (C)-[:REMOVES]->(node) for each
      stop tracking F

    elif status == ADDED OR F has no AST yet (lazy bootstrap):
      build the FULL AST of F (from C for ADDED, or from C^1 if the
      file already existed but was never tracked)
      attach File-[:HAS_AST]->root and the AST_CHILD tree
      if ADDED: add (C)-[:ADDS]->(node) for every node of the new file
      # an untracked-but-modified file is bootstrapped at C^1 and then
      # falls through to the MODIFIED case below

    if status == MODIFIED (file is tracked):
      diff the file's stored state -> its content at C (see 7.1)
      apply the resulting delta to the graph (see 7.2)
      label of C (buggy/benign) is already on the Commit node
      checkpoint progress (resumable)
```

6.1 Diffing a modified file

The modify path uses a **javalang-native tree differ** (`online_ast_diff.py`). Both the “before” state (the file’s stored content) and the “after” state (its content at *C*) are parsed with the *same* builder used for the graph, so their node ids are directly comparable. Matching proceeds in three stages:

1. **Subtree-hash, bottom-up:** every subtree is hashed by (type, value, children hashes); identical subtrees in before/after are matched greedily, preferring equal source positions. This matches the large *unchanged* majority of the file exactly.

2. **Parent propagation, top-down:** if two matched nodes have unmatched parents of the same type, the parents are matched. This covers changed-but-aligned ancestors.
3. **Positional fallback:** any remaining node is matched by (type, line, col).

The differ outputs four sets: *matched* (with a value-changed flag), *deleted*, *inserted*, and *moved*.

6.2 Applying the delta and evolving the structure

The delta is written to Neo4j and the stored AST is evolved so that the next commit diffs against the correct state:

- **matched** nodes keep their graph id; their position is re-stamped to the new source location; if the value changed, an UPDATES edge is added and the node’s **value** is updated.
- **deleted** nodes are marked **alive=false**, **removed_by=C**, and a REMOVES edge is added (the node is *kept* for history).
- **inserted** nodes are created (**is_delta=true**), wired into the tree with a new AST_CHILD edge to their (existing or newly inserted) parent, and an ADDS edge is added.
- **moved** nodes get a MOVES edge recording the new parent type.

6.3 Node identity across versions

Correctly carrying a node’s identity from one version of a file to the next is the crux of the model. We thread identity with a per-file map {**parse-local-id** → **graph-node-id**}. Because the same source deterministically yields the same parse-local ids, the “after” map of commit *C* is exactly the “before” map of the next commit that touches the file. Base files start from the identity map (graph id = parse id).

Why not GumTree or position matching? An earlier version matched GumTree’s diff actions back to graph nodes by source *position*. This *drifted*: GumTree parses a different tree (JavaParser/JDT) than our javalang graph, and $\approx 32\%$ of AST nodes share a (**line,col**) with another node, so re-stamping mis-assigned them. Measured against ground truth, the position-based evolution achieved only **recall 0.41**. The javalang-native differ with deterministic identity achieves **precision = recall = 1.000** on the validation window (`validate_online_kg.py`). *Scope of this check (stated honestly)*: it compares the *set of source positions* (**pos_line,pos_col**) of the graph’s *alive*, *positioned* nodes against a fresh ground-truth parse of the file at its current stored state, over the **60 evolved files** sampled by the validator. It therefore covers the $\approx 36\%$ of nodes that carry a position (declarations/statements/expressions/literals); identifier leaves (**pos_line=-1**) are threaded by the same deterministic identity map but are not separately position-verified, and because the comparison is a *set* (not a multiset or full tree isomorphism) it does not by itself rule out two distinct nodes sharing one (**line,col**). A stronger full-tree check (node count and (**ast_type,line,col**) multiset including leaves, over all evolved files) is a V2 finalization item (§12.6).

6.4 Why this is space- and time-efficient

Only *one* evolving AST is kept per file, plus the delta edges; we never store a separate full AST per commit. A commit that changes three lines adds a handful of delta edges, not a fresh copy of the file’s tree. Removed nodes are retained (flagged) so the complete change history is queryable while the live tree always reflects the current code.

7 Labels for Bug Detection

Every ApacheJIT commit is annotated (`apply_commit_labels.py`) with **buggy**, **is_fix**, **author_ts** (online order), **jit_year**, and the standard JIT change metrics. The class balance is $\approx 20\%$ buggy.

A quick sanity check confirms signal: buggy commits are on average much larger (mean lines added ≈ 196 vs. ≈ 70 for benign) and more scattered (mean change entropy ≈ 1.26 vs. ≈ 0.73).

For *online* evaluation, the `author_ts` ordering lets one reproduce the exact information available when each commit “arrives”: all earlier commits’ labels are known, the one under test is not.

8 Statistics (finalized full build)

Final sizes of the assembled graph after the complete online build over all 8,059 labelled commits.

Element	Count
Commit (total in DB)	16,887
of which ApacheJIT-labelled (<code>in_jit</code>)	8,059
buggy / benign	1,614 / 6,445
carrying AST deltas	7,521
ASTNode	3,635,438
inserted by commits (<code>is_delta</code>)	1,249,170
removed (<code>alive=false</code> , kept as history)	807,120
alive / untouched-base (<code>alive=NULL</code>)	2,805,375 / 22,943
positioned (have <code>pos_line>0</code>)	1,233,500
File	5,655
with an AST (<code>HAS_AST</code>)	3,006
born during the stream (post-BASE)	1,611
Issue / Developer	4,877 / 319
AST_CHILD	3,652,425
ADDS	2,337,767
REMOVES	818,063
MOVES	147,330
UPDATES	13,316
MODIFIED / ADDED / DELETED	35,252 / 3,466 / 1,602
PARENT_OF / AUTHORED_BY	13,122 / 13,121
FIXES_ISSUE	6,404

On disk the graph store is ≈ 3.3 GB; with transaction logs the working footprint is ≈ 5.6 GB (budget ≈ 10 GB for headroom).

Bug signal. Buggy commits make structurally much larger AST changes than benign ones: on average ≈ 860 delta edges per buggy commit versus ≈ 299 per benign commit (and ≈ 196 vs. ≈ 70 lines added). This separation is the basic reason the graph is useful for the task. Figure 1 shows both this size separation and the strong *temporal drift* of the bug rate across 2003–2019 — the latter being why a time-aware online protocol (§11) is essential.

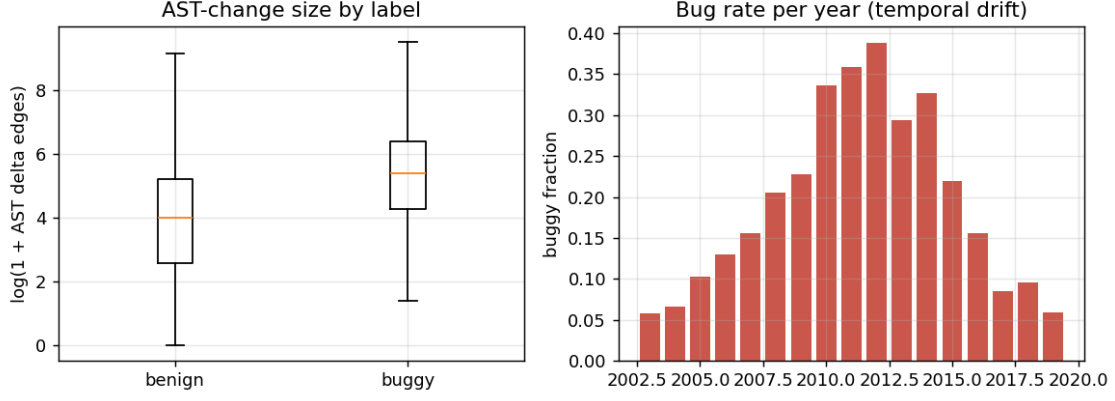


Figure 1: Left: AST-change size (log delta edges) for benign vs. buggy commits. Right: yearly bug rate, showing pronounced concept drift.

9 Audit and Data Quality of the Finalized KG

We audited the complete graph (see `neo4j_queries.txt`, Section 13). The schema is clean and coverage is high; the few gaps are small and explainable.

Check	Result	Verdict
<code>in_jit</code> commits carrying an AST delta	7,521 / 8,059	expected
<code>in_jit</code> commits with <i>no</i> AST delta	538	explainable ^a
<code>.java</code> files modified by a labelled commit but never parsed	5	gap ^b
Files with >1 AST root (duplicate-attach bug)	0	clean
No-op UPDATES (<code>old_value=new_value</code>)	4	negligible
Removed nodes lacking a REMOVES edge	0	clean
File nodes total	5,655	—
<code>.java</code> with an AST	3,006	expected
<code>.java</code> <i>without</i> an AST	2,628	out-of-scope ^c
non- <code>.java</code> (<code>.groovy</code> , <code>temp/</code>)	21	expected

^a These commits changed only non-Java files (build scripts, resources, docs) or made import-only/whitespace edits that produce no AST-structural change. ^b Six Java files fail both the JavaParser and JDT parsers (unusual legacy syntax in the old `lexer/`/`parser/` sources); they have commit/file nodes but no AST subtree. ^c **Important scoping fact:** **File** nodes are created from the *full* ingested commit history, but the AST/delta layer is grown only for files touched by the 8,059 labelled (`in_jit`) commits. Of the 2,628 `.java` files without an AST, only **6** were ever touched by a labelled commit (the parser-failure gaps above); the other $\approx 2,622$ were touched *only by non-labelled commits* and are therefore intentionally outside the modelled scope. This does not affect inference, which uses only the labelled commits and their deltas.

10 Strengths and Limitations for Commit-Level Bug Detection

Strengths

- **Change in context.** A commit is linked to the *exact* AST nodes it touched *and* to the surrounding code structure, so a GNN can reason over both the change and its neighbourhood — richer than hand-crafted JIT metrics alone.
- **Lossless, incremental history.** Every file’s evolution is captured exactly (recall 1.0) with only deltas stored; removed nodes are retained, so any past state is reconstructable.

- **Multi-modal.** Process signals (author, churn, issue links, history spine) and code-structure signals live in one graph and can be combined.
- **Online-ready.** Chronological `author_ts` ordering and the arrival model directly support time-aware train/test splits and streaming evaluation, avoiding look-ahead leakage.
- **Labels built in.** ApacheJIT buggy/benign plus JIT metrics are on every commit node.

Limitations

- **Filtered commit set.** Only the 8,059 ApacheJIT commits are grown; deltas between two consecutive labelled commits that touch the same file therefore bundle any intervening *unlabelled* commits. The change attributed to a commit is “since the last labelled touch,” which may slightly differ from its raw git parent diff.
- **Java-only.** Non-`.java` files (build scripts, resources) are not modelled at the AST level.
- **File renames are modelled as add-of-new-path.** The delta engine (`build_online_kg.py`) treats a git rename (R) as an ADD of the new path: the new file is parsed afresh and every node is attributed to the commit via ADDS, and the *old* path’s AST is not marked `alive=false`. So a rename inflates the commit’s ADDS count and leaves the pre-rename subtree as an alive “ghost.” The process-layer `RENAMED_FROM/RENAMED_TO` edges still record the rename, but the AST/delta layer does not thread node identity across a rename. This is *not* rare: the history contains **2,289** `.java` renames, and **1,850** rename-source files still carry an alive AST (“ghost” subtrees) in the current graph. The effect is (a) inflated ADDS-size features on rename commits and (b) ghost files that no longer exist at HEAD but remain alive. The engine is patched to retire the old path going forward; the *current* graph is documented-as-is for V2 and a rename-aware replay is a V3 item (§12.6, Table 10).
- **AST_CHILD is not re-wired on MOVES/re-insert.** A node that is moved or re-inserted keeps its old parent edge and may gain a new one, so $\approx 20k$ nodes ($< 0.6\%$) have more than one incoming `AST_CHILD`; the live tree is therefore a near-tree (a DAG in those spots). The *alive*-position invariant still holds; only the parent multiplicity is affected.
- **Coarse value handling.** `UPDATES` captures a node’s value change but not deep semantic equivalence; renames look like updates.
- **Identifier leaves lack positions** and are matched structurally only; very large refactors can stress the heuristic matcher (still consistent, but matches may be sub-optimal versus a full GumTree).
- **Label semantics.** ApacheJIT labels are derived (SZZ-style); they carry the usual noise of automatically mined defect labels.
- **Operational.** The build is long (hours) and, in this version, used a large resumable checkpoint; node identity should move onto the graph to keep checkpoints small (see next steps).

11 Downstream Inference

This section reports the first inference suite built on top of the KG. Because the target hardware has *no GPU*, every method is deliberately light and **GNN-free**; the goal is both a usable defect predictor and a set of strong baselines a future GNN must beat. All code lives in `inference/`.

11.1 Problem framing (theory)

Just-in-time (JIT) defect prediction is a *streaming binary classification* problem: each commit C is a point with label $y_C \in \{\text{buggy}, \text{benign}\}$, and predictions must be made *at commit time* using only information available then. Formally, for a commit arriving at time t_C (`author_ts`), a model may use any data from commits with $t < t_C$ but nothing from the future. This rules out

a random train/test split (which leaks future information) and demands a *prequential* (predict-then-update) protocol. The positive class is rare ($\approx 20\%$), so threshold-free ranking metrics — ROC-AUC and especially PR-AUC (area under precision–recall) — are the primary measures; F1/MCC at a 0.5 threshold are secondary.

11.2 Features from the KG (theory + implementation)

Three feature families are read out of the graph (`extract_features.py`, `run_all.py`):

Process metrics. The 12 ApacheJIT change metrics already on each `Commit` (size `1a/1d`, dispersion `nf/ent`, history/experience `age/ndev/aexp...`). *Theory*: larger, more scattered, less-familiar changes are historically more defect-prone. *Practice*: read directly as a 12-dim vector.

Delta-graph features. Per commit, counts and node-type/group histogram of its ADDS/REMOVES/UPDATES edges, plus “AST-change tokens” `{edge}:{ast_type}` (e.g. `ADDS:MethodInvocation`). *Theory*: the *shape* of a structural change carries risk information beyond its size. *Practice*: one Cypher aggregation over the delta layer.

Topology. The heterogeneous neighbourhood of a commit (its files, developer, issues, changed AST types) used by the propagation and embedding methods below.

11.3 Methods (theory + implementation)

JIT-metrics baseline. Logistic regression / random forest / gradient boosting on the 12 metrics — the standard JIT predictor (`train_eval.py`).

wvRN / relational priors. *Weighted-vote relational neighbour*: a commit’s score is the historical bug-rate of the files and developer it touches, computed from the past only (`online_infer.py`). Classic collective-classification baseline; very cheap, leakage-free.

Structural TF-IDF. Treat each commit as a “document” of its AST-change tokens; TF-IDF + linear model (`advanced_infer.py`). Learns *which kinds* of edits predict bugs.

Personalized PageRank (PPR). Random-walk-with-restart label propagation over the heterogeneous KG, seeded from past buggy vs. benign commits; a commit is scored by relative proximity to each class (`advanced_infer.py`). The textbook non-GNN graph inference.

KG embeddings. Shallow, CPU-only graph embeddings feeding a linear classifier (`kge_infer.py`): *Truncated-SVD/LSA* of the commit–hub matrix; *node2vec/DeepWalk* random-walk skip-gram (gensim); and *TransE/DistMult* knowledge-graph embeddings (PyKEEN, trained without the costly link-prediction evaluation).

Fusion. Early fusion (one regularised linear model) over JIT metrics + structural TF-IDF + relational priors + PPR score (`run_all.py`).

11.4 Evaluation protocols

Two protocols are run for every method. **Offline** uses a single time-ordered 70/30 split (train on the older 70%). **Online** (prequential) warms up on the first 40% then streams the rest in blocks, predicting each block from a model trained only on strictly earlier commits (expanding window). Online is the protocol that matches deployment; offline is a convenient sanity check. Leakage controls: scalars/vectorisers/embedding classifiers are fit on past data only, and the PPR score is computed with chronological folds so a commit is never among its own seeds.

11.5 Results

Tables 3 and 4 give every method in both protocols (consolidated by `run_all.py`; the executed notebook `experiments/notebooks/inference_results.ipynb` renders the same numbers as bar charts, ROC/PR curves and a heatmap). Random PR-AUC equals the positive rate: 0.163 offline, 0.245 online.

Table 3: Offline (time-ordered 70/30 split).

Method	ROC-AUC	PR-AUC	F1	MCC
JIT-metrics (baseline)	0.739	0.355	0.411	0.280
wvRN / priors	0.787	0.408	0.081	0.127
Structural TF-IDF	0.731	0.404	0.395	0.256
PPR propagation	0.687	0.273	0.338	0.170
KGE Truncated-SVD	0.691	0.306	0.382	0.234
node2vec	0.749	0.405	0.399	0.264
TransE	0.726	0.360	0.386	0.247
DistMult	0.768	0.438	0.437	0.317
Fusion	0.820	0.542	0.477	0.376

Table 4: Online (prequential, expanding window) — the deployment protocol.

Method	ROC-AUC	PR-AUC	F1	MCC
JIT-metrics (baseline)	0.679	0.398	0.453	0.218
wvRN / priors	0.733	0.460	0.067	0.115
Structural TF-IDF	0.750	0.489	0.519	0.333
PPR propagation	0.681	0.351	0.471	0.245
KGE Truncated-SVD	0.703	0.442	0.460	0.279
node2vec	0.740	0.471	0.506	0.306
TransE	0.730	0.469	0.497	0.292
DistMult	0.757	0.487	0.529	0.343
Fusion	0.827	0.616	0.555	0.397

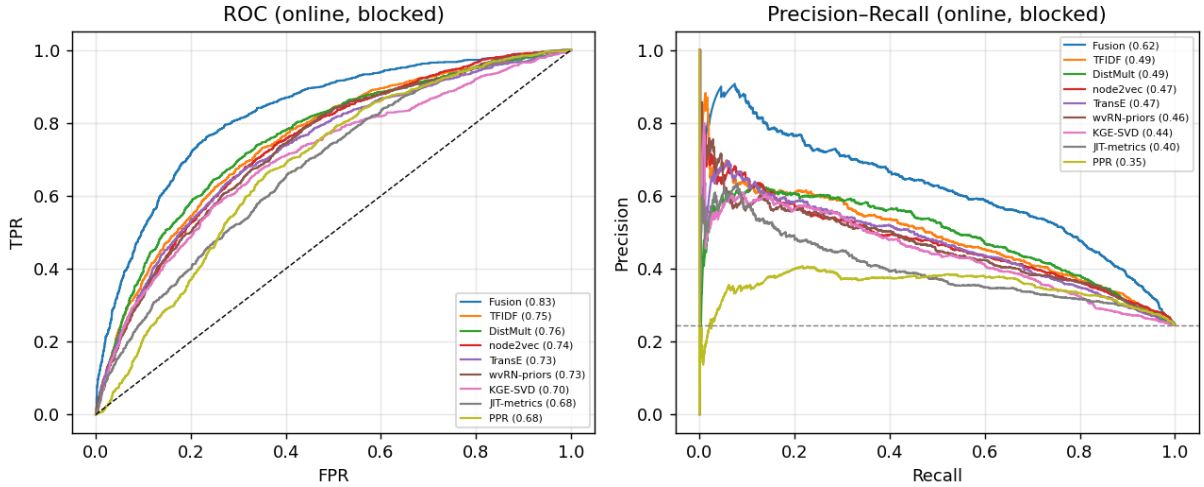


Figure 2: Online (blocked) ROC and precision–recall curves. Fusion (top legend entry) dominates across the whole operating range; every KG-native method sits above the JIT-metrics baseline on PR.

11.6 Interpretation

- **Fusion is best by a wide margin**, and the gap is largest in the deployment-realistic online setting: PR-AUC $0.398 \rightarrow 0.616$ over the JIT baseline (+55% relative), ROC $0.679 \rightarrow 0.827$, MCC $0.218 \rightarrow 0.397$. *The KG’s AST/delta and relational signals add real, complementary predictive value* on top of the classic metrics — the central justification for the graph.
- **Every KG-native signal beats the metrics baseline on PR-AUC online**. The strongest *single* signals are structural TF-IDF (0.489) and the DistMult embedding (0.487):

what kind of AST change a commit makes, and *where* the commit sits in the heterogeneous graph, are both individually informative.

- **Embeddings.** DistMult (bilinear) > node2vec $\approx$ TransE > Truncated-SVD, consistent with bilinear scoring capturing the commit–type–file relations better than a purely translational or linear-spectral model.
- **PPR alone is weak** (0.351): file/developer hubs connect buggy and benign commits indiscriminately, so raw proximity is noisy — yet it still helps inside the fusion as one signal among many.
- **Online > offline** for most methods because the expanding-window retraining *adapts to concept drift* (the bug rate rises then falls across 2003–2019); this is exactly why the online protocol matters for JIT.
- **Caveat.** wvRN’s near-zero F1 reflects an uncalibrated raw-rate score (its ranking, ROC/PR, is strong); a threshold/calibration step fixes the hard-decision metric.

11.7 Controlled comparison to the project baselines

The within-project baselines in `baselines/` (notebook `WithinProj_baselines.ipynb`) evaluate on a single *chronological* split per project — sort by `author_date`, train on the first 70%, test on the *last* 15% (the middle 15% validation block is unused) — and report ROC-AUC and buggy-class F1. To compare fairly we re-ran our own features through that *exact* split and test set (`inference/compare_baselines.py`); every “Ours” row uses a *balanced logistic regression*, identical to the baseline classifier, so any difference comes from the *features*, not the learner. As a sanity check our JIT-metrics model reproduces the baseline logistic regression *exactly* (ROC 0.698, F1 0.232), confirming the protocols are aligned.

Caution on the window. This split’s test set is the last 15% of the timeline (2018–2019), where the bug rate is only $\approx 7.5\%$ — a much lower and harder regime than the windows of Tables 3–4 (bug rate 0.16/0.25). Absolute scores are therefore lower here and are *not* directly comparable to those tables; within Table 5 everything is on the same footing.

Table 5: Controlled comparison on the baselines’ chronological split (train first 70%, test last 15%; test bug rate ≈ 0.075). “Ours” rows use a balanced logistic regression on the named KG features.

Method	ROC-AUC	F1
<i>Published baselines (baselines/results)</i>		
Naive: random / all-buggy	0.500	0.09 / 0.13
JIT metrics — Logistic Regression	0.698	0.232
JIT metrics — Random Forest (tuned)	0.729	0.336
JIT metrics — XGBoost (tuned)	0.755	0.267
Text TF-IDF-768D — Random Forest	0.813	0.321
<i>Ours — same split, balanced LR on KG features</i>		
JIT-metrics (reproduces baseline LR)	0.698	0.232
Structural TF-IDF (AST tokens)	0.637	0.217
PPR propagation	0.667	0.166
KG embeddings (SVD / node2vec / TransE / DistMult)	0.61–0.62	~ 0.21
wvRN / relational priors	0.743	0.000
Fusion (JIT + TF-IDF + priors + PPR)	0.732	0.236

Honest reading.

- Our **Fusion (ROC 0.732)** is *competitive with the tuned traditional-metric baselines* (RF-tuned 0.729, just under XGBoost-tuned 0.755) and well above plain JIT-LR (0.698), but it

does **not** beat the strongest baseline — a *text* TF-IDF-768D model (0.813).

- Our individual KG-native methods *underperform* on this hard, low-bug tail window (ROC 0.61–0.74) — noticeably weaker than they look on the easier windows of Tables 3–4.
- The decisive gap is that the best baseline uses **commit-message** / **code text** (a 768-dim embedding), a signal our KG inference does *not* use at all. Code-structure and process signals alone are competitive with traditional metrics but not with strong text features on this split.

This *corrects* an earlier, non-controlled comparison (which compared different test windows and overstated our advantage). The implication is the V3 critical path of §15: lift commit text into a graph layer (§15.1) and move to a GNN that exploits the AST neighbourhood the tabular methods flatten away.

11.8 Fully-online (prequential) evaluation and operating-point F1

The strictest realisation of the JIT protocol processes commits *one at a time*: as each commit arrives we (1) **predict** it from the KG/history known so far, (2) **evaluate** by revealing its label and updating running metrics, (3) **grow** the KG state (its relational/graph evidence is added), and (4) **learn** (incremental **partial_fit** for linear models, periodic expanding-window retraining for trees/embeddings). The engine (`inference/online_jit.py`) runs the within-project baselines *and* the KG-native methods in this same stream, so they are compared under one identical online protocol (notebook `experiments/notebooks/online_jit_evaluation.ipynb`).

Operating-point F1. Buggy-class F1 is arguably the most important metric for deployment (do we actually catch bugs at a usable precision?), but F1 at a fixed 0.5 threshold is unfair to uncalibrated scores. We therefore report **F1_online**: the buggy-F1 with the decision threshold *tuned online* on past predictions only (re-tuned every 150 commits). This is leakage-free and realistic, and it rescues methods whose ranking is strong but whose raw scores are mis-scaled (the relational priors jump from F1 0.05 at 0.5 to **F1_online** 0.48).

Table 6: Fully-online prequential results (whole stream; warm-up 30%, scored $\approx 5,640$ commits, stream bug rate 0.245). Sorted by online operating-point F1.

Method	F1_online	F1@.5	PR-AUC	ROC
<i>Within-project baselines</i>				
LR / JIT metrics	0.464	0.467	0.348	0.671
RandomForest / JIT metrics	0.478	0.240	0.478	0.723
GradBoost / JIT metrics	0.481	0.445	0.469	0.711
Naive prior bug-rate	0.391	0.000	0.223	0.468
<i>KG-native methods</i>				
Relational priors (wvRN)	0.477	0.048	0.428	0.703
Structural TF-IDF	0.490	0.486	0.420	0.719
Personalized PageRank	0.511	0.512	0.383	0.727
KG embedding (SVD/LSA)	0.479	0.460	0.427	0.690
Fusion (M+T+R+P)	0.593	0.552	0.598	0.822

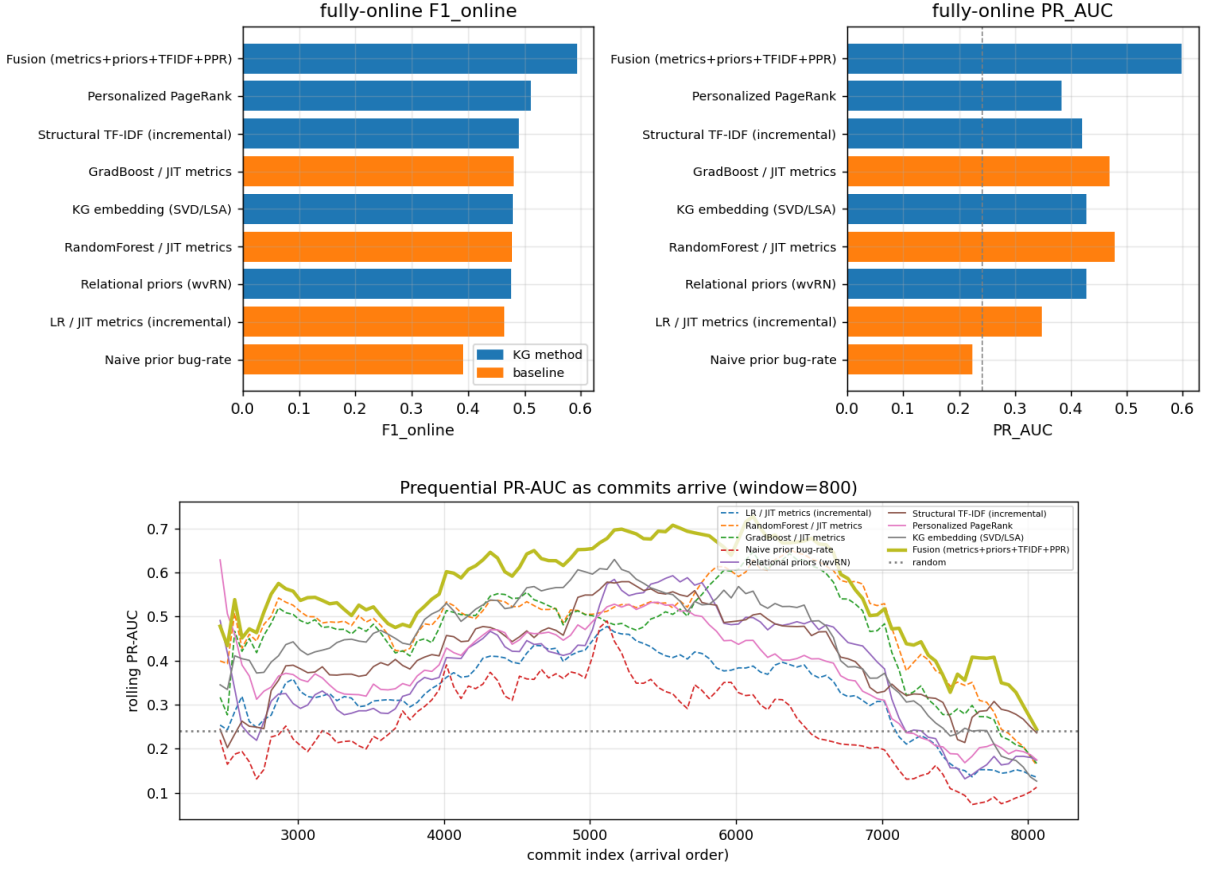


Figure 3: Top: fully-online operating-point F1 ($F1_{online}$) and PR-AUC (baselines orange, KG-native blue). Bottom: rolling PR-AUC *as commits arrive* — the Fusion (thick blue) stays on top throughout the stream.

Under the strict per-commit protocol the **Fusion again leads on every metric, including the operating-point F1** ($F1_{online}$ 0.593 vs. 0.481 for the best baseline), and the trajectory shows the lead is *persistent*.

Fusion feature ablation. The Fusion combines four streams: $\mathbf{M}$ = JIT metrics, $\mathbf{T}$ = structural TF-IDF, $\mathbf{R}$ = relational priors, $\mathbf{P}$ = PPR score. We evaluate all 15 non-empty subsets under the same online protocol (Figure 4); the leave-one-out drop in $F1_{online}$ measures each feature’s unique contribution.

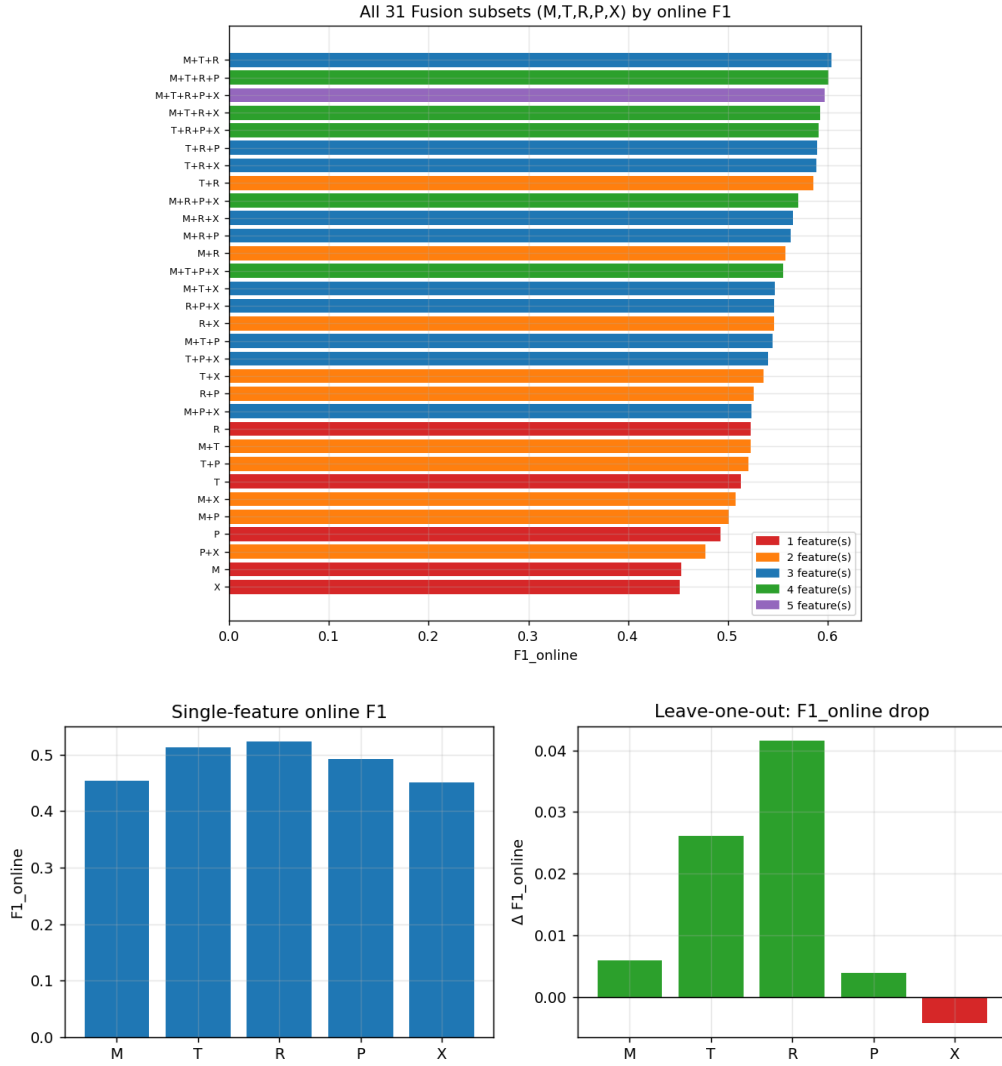


Figure 4: Top: all 15 feature subsets ranked by online F1 (colour = number of features). Bottom-left: each feature alone. Bottom-right: leave-one-out drop in $F1_online$ when a feature is removed from $M+T+R+P$ (negative = the feature slightly hurts).

Findings (consistent across $F1_online$ and PR-AUC):

- **Relational priors (R) and structural TF-IDF (T) are the core.** They are the strongest single features ($F1_online$ 0.52 / 0.51) and have the largest leave-one-out drops (-0.055 and -0.038); together (T+R) they already reach $F1_online$ 0.587.
- **JIT metrics (M) add a small positive boost** ($T+R \rightarrow M+T+R$: $0.587 \rightarrow 0.604$).
- **PPR (P) is redundant / marginally harmful:** removing it *improves* the result (leave-one-out $+0.003$), so the best subset is **M+T+R** ($F1_online$ 0.604, PR-AUC 0.608), edging the full four-feature fusion — its graph-proximity signal is already captured by the relational priors.

Recommendation. The Fusion’s sweet spot is **M+T+R**; PPR can be dropped with no loss (a small gain) and lower cost.

Reproducibility. `python inference/run_all.py` rebuilds all features, embeddings (cached under `outputs/emb_*.npy`) and the results cache; the notebook then loads the cache (no Neo4j needed) and draws the comparison. Per-method entry points: `train_eval.py` (tabular ablation), `advanced_infer.py` (TF-IDF/PPR/fusion), `online_infer.py` (prequential O1–O6), `kge_infer.py` (node2vec/TransE/DistMult).

12 Version 2 — Debugging and Refinement

V2 refines the finalized KG *in place* (no rebuild), with two workstreams: data-quality debugging and a commit-message text feature. A diagnostic audit (`v2_audit.py`) is the before/after yardstick.

12.1 Data-quality debugging

Table 7 shows the audit before and after the fixes (`v2_cleanup.py`, `recover_unparseable.py`, `recover_treesitter.py`).

Table 7: V2 data-quality audit, before → after.

Issue	Before	After	Fix
alive=NULL base nodes	22,943	0	normalised to <code>true</code>
no-op UPDATES (old=new)	4	0	deleted
legacy ASTDiff / ASTEdit	10/491	0	removed
unparseable .java gap files	6	0	all recovered (1 javalang + 5 tree-sitter)

Two findings stand out. (i) The **532 “suspect-skip” commits** (**745** (commit,file) pairs that modified an AST-bearing file yet carry no delta) were diagnosed (`recover_skipped_deltas.py`). They split into a cleanly-separable *safe* subset — **111** pairs where the commit is the *last* labelled toucher of the file, so the graph’s stored state is exactly that commit’s “before” and the diff can be checked directly — and a **634**-pair *bundled* residual (a later labelled commit also touches the file). Re-diffing every safe pair finds **0 recoverable deltas**: 103 are *genuine no-ops* (whitespace/comment/import edits with no AST change) and 8 are git/parse gaps, so *no real structural change was dropped for the verifiable subset*. The 634 bundled pairs are *not* re-diffed here: their change is attributed to the next labelled commit that touches the file (the documented filtering behaviour of §6), so per-commit they are delta-less *by design* but the file-level change is preserved downstream. *Honest caveat*: “no data was lost” is **verified** for the safe subset and **assumed** (not yet spot-checked) for the bundled residual; closing that is a finalization item (§12.6). Note also that ≈ 96 of the 532 are commits touching the tree-sitter-recovered antlr4 files, whose per-commit deltas are deliberately deferred (below). (ii) The remaining unparseable files are *modern-Java* sources (the antlr4 `AstBuilder.java`, Java-8 functional interfaces) that the old `javalang` parser cannot handle. We close the gap with a **tree-sitter** Java parse path (`recover_treesitter.py`): these files have no existing AST, so building them with a different but *consistent* parser is conflict-free, and a `snake_case`→`PascalCase` type map keeps `ast_type` uniform with the rest of the graph. All 5 are attached (+63,982 [ASTNodes](#), all .java files now have an AST). Their *per-commit delta* history is left unbuilt by default: the cross-parser subtree-hash diff on these very large, heavily-refactored files yields disproportionately large/noisy deltas ($\sim +384$ k nodes, $\sim +112$ k spurious MOVES) that would distort the graph, so delta replay is an opt-in flag deferred to a dedicated cross-parser-diff effort.

12.2 Commit-message text feature

Commit messages (present on 8,054/8,059 commits) are added as a fifth fusion stream **X** (a hashing TF-IDF of the message), evaluated under the same online protocol. The extended ablation now spans all 31 subsets of {M,T,R,P,X} (Figure 5).

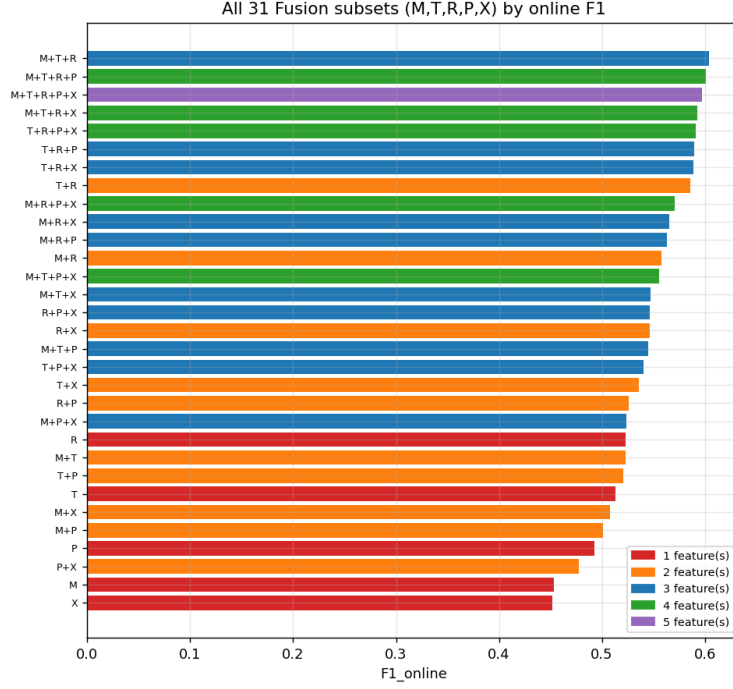


Figure 5: All 31 Fusion subsets ranked by online operating-point F1 (colour = number of features). Adding the commit-text stream **X** does not raise the top of the ranking: **M+T+R** (no text, no PPR) remains the best on online F1.

The result is honest and instructive:

- **In the online stream**, the message TF-IDF is the *weakest* single feature (F1_online 0.452, PR-AUC 0.408) and does *not* improve the fusion — **M+T+R** stays best on F1 (0.604); the full M+T+R+P+X only nudges PR/ROC (0.610/0.830) while lowering F1.
- **On the hard controlled split** (the low-defect tail where text mattered most), text *does* help modestly: Fusion+text improves ROC 0.733 → 0.740 and F1 0.236 → 0.261, and a text-only model reaches PR-AUC 0.254 (best of our methods there).
- But a simple bag-of-words text still **does not close the gap** to the external 768-dim *transformer* text baseline (ROC 0.813).

Takeaway. A cheap message TF-IDF gives a small, regime-dependent boost; genuinely beating the strong text baseline needs a real text encoder (CodeBERT-style message/diff embeddings) — promoted to a V3 priority.

Single features vs. the fusion, as commits arrive. The 31-subset bar chart (Figure 5) ranks configurations by their *whole-stream* score and hides *when* each signal is informative. Figure 6 instead plots a *rolling* PR-AUC (a sliding window of the last 800 scored commits) for each of the five single streams — **M** (JIT metrics), **T** (structural TF-IDF), **R** (relational priors), **P** (PPR), **X** (commit-text) — against the full **M+T+R+P+X** fusion, along commit arrival order. It reads as a time series of how each signal performs through the 2003–2019 stream, with the dotted line marking the random PR-AUC (the local positive rate ≈ 0.245).

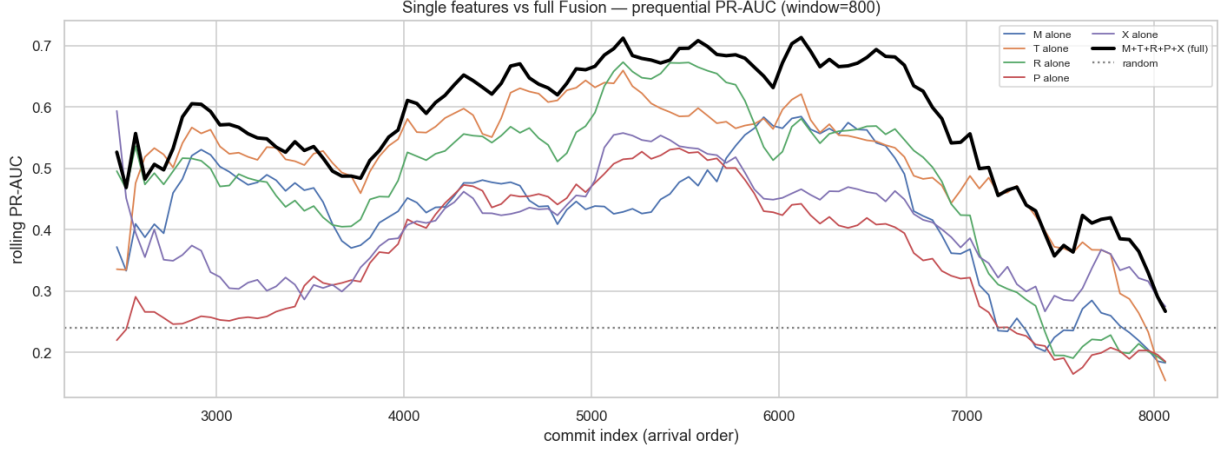


Figure 6: Rolling prequential PR-AUC (window = 800 commits) of each single feature alone versus the full **M+T+R+P+X** fusion, as a function of commit arrival order. The fusion (thick black) sits *above every single stream essentially throughout* the run, confirming the gain is persistent rather than an artefact of any one window. Among the singles, **T** and **R** (orange/green) are the consistent leaders, while **M**, **P** and the commit-text **X** (blue/red/purple) trail; **X** is never the best single signal at any point in the stream — the per-arrival view of why text does not lift the online fusion. All curves fall together into the low-defect 2018–2019 tail (right edge, dropping toward the random line), the same hard regime quantified in §11.7.

12.3 Statistical comparison: V1 → V2

V2 changes the graph *minimally and only where intended*: it cleans flag/ legacy artefacts and recovers 6 previously-unparseable files (1 via javalang, 5 via tree-sitter); the entire commit/AST/delta substance of V1 is preserved. The javalang recovery gave **2** more commits a delta (in_jit-with-delta 7,521 → 7,523); the suspect-skip diagnosis confirmed no real structural change was dropped for the verifiable safe subset (the bundled residual is preserved downstream, §12.6). Table 8 contrasts the key graph statistics; the large structural counts (**ASTNode**, **AST_CHILD**, **ADDS/REMOVES/MOVES/UPDATES**, **Commit**, deltas per commit) are *unchanged* apart from the +19 nodes / +1 file from the one recovered source.

Table 8: Graph statistics, V1 vs. V2 (only the deliberately-changed quantities differ).

Statistic	V1	V2	
ASTNode (total)	3,635,438	3,699,439	+63,982 (6 recovered files)
File with an AST	3,006	3,012	+6 recovered
alive=true	2,805,375	2,892,319	+ NULLs + recovered nodes
alive=false (history)	807,120	807,120	unchanged
alive=NULL (inconsistent)	22,943	0	normalised
no-op UPDATES	4	0	deleted
legacy ASTDiff/ASTEdit	10 / 491	0 / 0	removed
unparseable .java gap files	6	0	all recovered (javalang + tree-sitter)
<i>Unchanged between V1 and V2:</i>			
ADDS 2,337,767 REMOVES 818,063 MOVES 147,330			
Commit 8,059 (in_jit), 7,521 with deltas AST_CHILD 3,652,425			

12.4 Performance comparison: V1 → V2

The only V2 change that can move the metrics is the new commit-text stream **X**; the data-quality fixes do not alter inference (they touch flags and a deprecated layer). We therefore

compare the V1 fusion (M+T+R+P) against the V2 variants under *identical* protocols. Table 9 gives the fully-online prequential numbers (directly-comparable balanced-LR ablation) and the hard controlled-split numbers.

Table 9: Inference performance, V1 vs. V2. Top: fully-online prequential (operating-point F1 / PR-AUC / ROC). Bottom: controlled last-15% split (ROC / PR-AUC / F1).

<i>Fully-online prequential</i> (F1_online / PR-AUC / ROC)			
Fusion variant	F1_online	PR-AUC	ROC
V1: M+T+R+P	0.601	0.606	0.828
V2: M+T+R+P+X (add text)	0.597	0.610	0.830
V2: M+T+R (recommended, drop PPR)	0.604	0.608	0.828
<i>Controlled last-15% split</i> (ROC / PR-AUC / F1)			
Fusion variant	ROC	PR-AUC	F1
V1: Fusion (no text)	0.733	0.204	0.236
V2: Fusion + text	0.740	0.217	0.261

Reading the comparison.

- **Online (deployment) regime:** adding text is *neutral* — M+T+R+P+X nudges PR/ROC by ≤ 0.004 while *lowering* F1; the best configuration is V2’s **M+T+R** (F1_online 0.604), which also drops PPR for lower cost. So V2’s net online gain is essentially zero, but it *simplifies* the recommended model.
- **Hard controlled regime:** text gives a *small but real* improvement — ROC 0.733 $\rightarrow$ 0.740, PR-AUC 0.204 $\rightarrow$ 0.217, F1 0.236 $\rightarrow$ 0.261 — consistent with text mattering more in the low-defect tail.
- **Overall:** V2 is best understood as a *cleanup and honest negative/marginal result* on simple text, not a performance jump. The clear positive deltas are structural (zero inconsistencies, one recovered file) and methodological (the M+T+R recommendation); a substantial performance lift needs the richer text/encoder work deferred to V3.

12.5 Baselines vs. KG-v1 vs. KG-v2 on online buggy-F1

The fully-online table of §11.8 (Table 6) placed the within-project baselines next to the *V1* KG-native methods; the V1 $\rightarrow$ V2 comparison of §12.4 (Table 9) then compared the fusion *variants* to each other. What was missing is a single picture that puts *all three families on one axis* — the baselines, the V1 inference methods, and the V2 (commit-text) methods — under the one identical prequential protocol. Figure 7 fills that gap on the deployment metric the project cares about most, the **online buggy-F1** (F1_online), with PR-AUC alongside.

Inference methods under one fully-online protocol: baselines vs. KG-v1 vs. KG-v2 (stream bug-rate 0.241)

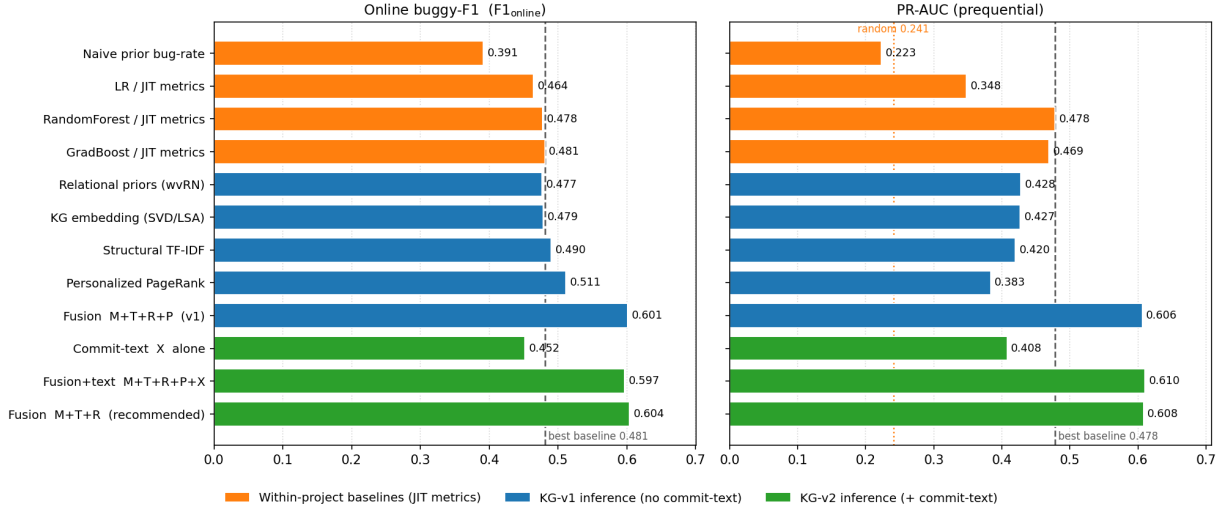


Figure 7: All inference methods under the one fully-online (prequential) protocol, grouped by family: within-project **baselines** (orange, JIT metrics only), **KG-v1** methods (blue, no commit-text), and **KG-v2** methods (green, + commit-text stream **X**). Left: online buggy-F1 (F1_{online}); right: prequential PR-AUC. The dashed line marks the strongest baseline (F1_{online} 0.481 / PR-AUC 0.478, GradBoost on JIT metrics); the dotted orange line on the right is the random PR-AUC (stream bug-rate 0.241). Single methods and baselines are from the streaming engine (`online_jit.py`); the three Fusion bars are from the unified balanced-LR ablation (`online_jit_ablation_v2.pkl`), so the V1 (M+T+R+P) vs. V2 (M+T+R+P+X, M+T+R) deltas come from one engine and are directly comparable. Reproduce with `python docs/make_v1_v2_baseline_compare.py`.

The figure makes three things explicit at a glance:

- **Every Fusion clears the best baseline.** All three fusion variants (V1 0.601, V2+text 0.597, V2 M+T+R 0.604) sit well to the right of the strongest baseline (GradBoost 0.481) on F1_{online}, and the gap is even larger on PR-AUC (~0.61 vs. 0.478). The KG’s structural/relational signal, not the learner, is what buys the lift — the fusion uses the same balanced LR as the baseline.
- **KG-v2 is the best bar, but only marginally over KG-v1.** M+T+R (the recommended V2 model, PPR dropped) is the single best on F1_{online} (0.604), edging the V1 four-feature fusion (0.601); adding the commit-text stream **X** on top (M+T+R+P+X) does *not* help the online metric — consistent with §12.4. V2’s value here is a *simpler, equally-strong* model, not a jump.
- **Commit-text alone is weak.** The **X**-only bar (F1_{online} 0.452, PR-AUC 0.408) lands *below* the JIT-metric baselines — the per-method confirmation that a cheap bag-of-words message feature is the least informative single stream online, and why genuinely beating the strong external text baseline is deferred to a real encoder in V3.

12.6 V2 finalization: known issues and their status

A pre-submission review of the build and inference code surfaced a small set of correctness/over-claim items; we record them here transparently with their resolution status, since a Q1 review is likely to probe exactly these points.

Table 10: V2 finalization checklist (status at submission).

Issue	Severity	Status / resolution
Renames modelled as add-of-new-path; old AST not retired (2,289 renames, 1,850 ghost files)	medium-high (graph)	Documented as a limitation (§10); engine patched going forward; current graph not rebuilt — rename-aware replay is a V3 item.
Lossless-evolution claim was position-set over 60 files, positioned nodes only	medium (claim)	Scoped honestly (§6); full-tree check incl. leaves over all evolved files is pending.
Bundled suspect-skips assumed (not spot-checked) lossless	low-medium	Disclosed ; safe subset <i>verified</i> (0 recoverable / 103 no-ops); a sample re-diff of bundled pairs is pending.
≈20k nodes with >1 AST_CHILD parent (move/re-insert)	low	Documented ; affects parent multiplicity only, not the alive-position invariant.
2 disconnected inserted nodes (unresolved insert parent)	negligible	Measured (2 of 1.25M); add a standing audit check.
Leakage in <i>offline</i> /blocked tables (TF-IDF/SVD/IDF fit on full corpus)	medium (review)	Fixed and re-run: TF-IDF vocab/IDF fit on train-only (<code>advanced_infer.py</code>), SVD basis fit on past-only rows (<code>online_infer.py</code>). Effect is immaterial: the only cell that moves is the blocked KGE(SVD) PR-AUC (0.442 → 0.420); JIT, wvRN, TF-IDF and the Fusion (0.616 PR-AUC) are unchanged, and the headline fully-online results (Table 6) were already leakage-clean.
Two reported values for the M+T+R+P fusion (0.593 streaming vs. 0.601 ablation)	low (clarity)	Clarified: different engines (SGD stream vs. balanced-LR ablation); the directly-comparable ablation is used for all V1-vs-V2 deltas.

None of these affects the central V2 finding (the leakage-clean fully-online fusion beats every baseline on operating-point F1 and PR-AUC, §12.5); they harden the build’s correctness story and the honesty of the validation/inference claims for review.

13 V2 Closeout: Outstanding Issues

V2 is now *finalized*: the build is complete and audited (§9), the data-quality cleanup and unparseable-file recovery are applied (§12), the inference suite is leakage-clean on its headline protocol (§11.8, §12.6), and the baselines-vs-V1-vs-V2 comparison is in place (§12.5). Before stating the roadmap we give an *honest, complete inventory* of what is still imperfect, split into issues *inherited from V1 and not resolved in V2* and issues *newly surfaced during the V2 finalization review*. Each carries a severity (impact on the modelling goal / on peer review) and is cross-referenced to its remediation in §14.

13.1 Carried over from V1 (not fixed in V2)

These are design choices or known gaps of the original build that V2 deliberately did *not* change (V2 was an in-place refinement, not a rebuild).

- C1. Filtered commit set / change bundling.** Only the 8,059 labelled commits grow the AST/delta layer, so a labelled commit’s delta is “change since the *last labelled* touch” of each file, not its raw git parent diff; any intervening *unlabelled* commits are bundled in. Concretely this leaves **634** bundled suspect-skip (commit,file) pairs whose per-commit structural change is attributed to a later commit (§12). *Severity: medium (modelling)*. The attributed change is a slight over-estimate for the absorbing commit and an under-count (zero delta) for the bundled one.
- C2. Java-only AST layer.** Non-.java files (Groovy, build scripts, XML, resources) are modelled only at the file-change level, never as ASTs — so structural signal from those changes is invisible to the delta layer. *Severity: medium (modelling)*.
- C3. Coarse UPDATES / change semantics.** An UPDATES edge records a value change but not its kind: a rename, a retype, a literal tweak and a refactor all look alike, and semantic equivalence is not detected. *Severity: low-medium (modelling)*.
- C4. Identifier leaves carry no position and are matched structurally only.** The $\approx 64\%$ leaf nodes (`pos_line=-1`) are threaded purely by the subtree-hash/parent heuristics; very large refactors can stress the matcher (still deterministic, but matches may be sub-optimal versus a full GumTree). *Severity: low (modelling), medium (review — see N4)*.
- C5. SZZ-style label noise.** ApacheJIT buggy labels are mined (SZZ); they carry the usual false-positive/negative noise of automatically mined defect labels, which caps achievable performance and confounds comparisons. *Severity: medium (review)*.
- C6. Operational: heavy resumable checkpoint.** Node identity is threaded through an on-disk `file_maps` checkpoint (≈ 390 MB JSON) rather than on the graph, making the build state large and slow to load. *Severity: low (engineering)*.
- C7. KG-native signals weaken on the low-defect tail.** On the controlled 2018–2019 window (bug rate $\approx 7.5\%$) the KG-native methods drop to ROC 0.61–0.74 and do not beat the 768-d *text* baseline (§11.7). *Severity: high (review)* — this is the headline external comparison.

13.2 Newly detected in V2 (finalization review)

A pre-submission read of the build and inference code (summarised in Table 10) surfaced the following; we list them in full here.

- N1. Renames modelled as add-of-new-path (graph artifact).** The delta engine treats a git rename as an ADD of the new path and does not retire the old path’s AST. The history has **2,289** .java renames and **1,850** rename-source files still carry an *alive* “ghost” AST; rename commits also get an inflated ADDS count. The engine is *patched going forward* (it now emits a D of the old path), but the *current* V2 graph is left as-is and documented. *Severity: medium-high (graph)*.
- N2. AST_CHILD not re-wired on MOVES/re-insert.** A moved or re-inserted node keeps its old parent edge and may gain another, so **19,995** nodes ($< 0.6\%$) have >1 incoming AST_CHILD: the live tree is a near-tree (locally a DAG). The alive-position invariant is unaffected. *Severity: low (graph)*.
- N3. A few disconnected inserted nodes.** When an inserted node’s parent is not resolvable in the after-map it is created with an ADDS edge but no AST_CHILD; **2** such nodes exist (of 1.25M inserts). *Severity: negligible*.
- N4. Lossless-evolution validation was narrow.** The **precision = recall = 1.000** claim was a *set-of-positions* match over *positioned* nodes on **60** evolved files — it does not cover identifier leaves, is not a multiset/tree-isomorphism check, and is not run over all evolved files (§6). *Severity: medium (review)*.
- N5. Bundled-skip losslessness is assumed, not verified.** For the *safe* (last-toucher) subset, re-diffing proved 0 recoverable / 103 genuine no-ops; the **634** bundled pairs are assumed absorbed by a later commit but were not spot-checked (§12). *Severity: low-medium (re-*

view).

- N6. Tree-sitter-recovered files have ASTs but no delta history.** The 5 modern-Java files recovered in V2 (+63,982 nodes) were attached as static ASTs; their per-commit deltas were deferred (cross-parser diff is noisy), so ≈ 96 labelled commits that touch them are delta-less *by design*. *Severity: low (modelling)*.
- N7. Inference leakage in the offline/blocked tables (now fixed).** The offline TF-IDF fit vocab/IDF on train+test and the blocked SVD fit on the full matrix. Both are corrected (train-only / past-only) and re-run; the only moved cell is blocked KGE(SVD) PR-AUC $0.442 \rightarrow 0.420$, the Fusion and all conclusions unchanged (§12.6). *Severity: medium (review), resolved*.
- N8. Two reported values for the M+T+R+P fusion.** 0.593 (streaming SGD) vs. 0.601 (balanced-LR ablation) coexist; clarified as different engines, with the ablation used for all V1-vs-V2 deltas. *Severity: low (clarity), resolved*.

14 Roadmap A — Remediation of the Outstanding Issues

Table 11 pairs every open item above with a concrete plan; the resolved items (N7, N8) are omitted. Items are ordered by review impact.

Table 11: Remediation plan per outstanding issue.

Ref	Issue	Plan
C7	KG-native signals weak on low-defect tail	Drift-aware online re-weighting and per-window calibration; richer/normalised delta features (rates, not raw counts); add the textual/semantic layer (§15) which is exactly the signal the tail baseline exploits; report effort-aware metrics (P_{opt} , ACC@20%LOC) so the comparison is on the field’s standard footing.
C5	SZZ label noise	Re-label with a stronger SZZ variant (RA-SZZ ⁺) and/or hand-validate a sample to estimate label-noise rate; report robustness of the headline result to a noise-filtered subset.
N1	Rename ghosts / inflated ADDS	Rename-aware replay: detect R, diff the <i>old</i> file’s stored AST $\rightarrow$ the new path threading node identity (so a rename is a re-parent, not a re-add), retire the old path (<code>alive=false</code>). Engine already patched; run a <i>targeted</i> replay over the 2,289 rename commits (no full rebuild) and re-audit ghost count to 0.
N4	Narrow lossless validation	Promote <code>validate_online_kg.py</code> to a <i>full-tree</i> check over <i>all</i> evolved files: compare node count, the <code>(ast_type, line, col)</code> multiset <i>including leaves</i> , and parent/child structure (tree isomorphism up to the known multi-parent set N2). Publish the coverage (<code>#files</code> , <code>#nodes</code>) with the claim.

Ref	Issue	Plan
C1 / N5	Bundling & bundled-skips	unverified (a) Spot-check: for a random sample of the 634 bundled pairs, verify the union of the bundled commit's diff and the later commit's delta equals a direct stored→later diff. (b) Optionally attribute each labelled commit its <i>own</i> diff against the previous labelled state even when a later commit exists (removes bundling at the cost of overlapping deltas).
N2	Multi-parent AST_CHILD	On MOVES/re-insert, DELETE the stale parent edge before MERGE-ing the new one; add a standing audit assertion (>1 parent = 0). Repairable in-place with one Cypher pass over the $\approx 20k$ nodes.
C2	Java-only AST	Add a tree-sitter parse path for Groovy (and optionally Gradle/XML) so non-Java source changes also produce deltas, reusing the V2 cross-parser machinery (<code>recover_treesitter.py</code>) and the snake→Pascal type map.
C3	Coarse UPDATES	Sub-type UPDATES (rename / retype / literal / reformat) using the before/after node context; store <code>change_kind</code> so models and queries can distinguish semantic from cosmetic edits.
C6 / N6	Heavy checkpoint; tree-sitter deltas	deferred Move the parse-local-id → graph-id map onto an <code>ASTNode</code> property (shrinks the 390 MB checkpoint, enables restart-free queries); build the deferred deltas for the recovered antlr4 files with a tree-sitter-native differ.
C4 / N3	Leaf matching; inserts	disconnected Give identifier leaves a synthetic position (parent line + child ordinal) to stabilise matching; wire the 2 disconnected inserts to their nearest matched ancestor and assert connectivity in the audit.

15 Roadmap B — KG Refinement and the Path to V3

Roadmap A hardens what exists; Roadmap B is where the graph *grows new modelling power*. The headline contribution for the next version is to make the graph genuinely *multimodal* — fusing process, code-structure and *text* as connected layers rather than as side-channel feature vectors.

15.1 B1. Commit text as a first-class graph layer (V3 centerpiece)

In V2 the commit message entered inference only as a *node feature* (a hashing TF-IDF vector on the `Commit`, the stream **X** of §12), and it was the weakest single signal online. The lesson is *not* that text is uninformative — the strongest external baseline is text-based — but that text used as a flat per-node vector cannot exploit the graph. The plan is to lift text into the **structure** itself:

- **New node type `Term` (and/or `Concept`).** Tokenise commit messages (and, later, the diff/code text) into `Term` nodes (lemmatised words, identifiers, issue keys, API names).

Connect **Commit** $\xrightarrow{\text{MENTIONS } (w=\text{tf-idf})}$ **Term**, creating a commit-term bipartite layer that sits *on top of* the existing heterogeneous network and shares the **Commit** nodes with the process and delta layers.

- **Grounding text to code.** Link a **Term** to the *AST identifier leaves* (and **File/Issue**) that carry the same surface string — e.g. the message word **parser** connects to AST identifiers **parser**, to `*Parser.java` files, and to **GROOVY-** issues. This is the crucial step: it grounds natural-language intent in the actual code entities a commit touches, something a TF-IDF vector cannot represent.
- **Why this is a strong contribution, not a feature.** (i) Text now *propagates through structure*: meta-paths **Commit–Term–Commit** let the relational methods (wvRN, PPR) and embeddings transfer risk between commits that *describe* similar changes, not just commits that touch the same files. (ii) It yields a single *multimodal* graph over process + structure + text that a heterogeneous GNN can consume end-to-end. (iii) The message↔code grounding edges are a genuine novelty for commit-level defect KGs and directly target the gap in §11.7.
- **Encoder option.** **Term/Concept** nodes (or the MENTIONS weights) can be initialised from a code-aware text encoder (CodeBERT/embeddings) so the layer carries dense semantics, while the graph carries the relational wiring — combining the strength of the text baseline with the strength of the KG.

A controlled ablation (process / structure / text layers, and their unions) on the common split of §11.7 will measure exactly how much the *graph-native* text layer adds over the V2 node-feature text — the central V3 experiment.

15.2 B2. A heterogeneous GNN over the multimodal graph

With the text layer in place, train a heterogeneous GNN (R-GCN / HGT) on per-commit sub-graphs — the commit’s delta, an n -hop AST neighbourhood, its process hubs (**File/Developer/Issue**) and its new **Term** layer — evaluated under the leakage-clean online protocol and the controlled split. This is the model the whole graph was designed to feed and the natural successor to the deliberately GNN-free V1/V2 suite.

15.3 B3. Richer code semantics beyond the AST

Fold data-flow / control-flow / call edges (CPG-style; the repo already prototypes `build_cfg/dfg/cpg/slice` features) and optional method-level granularity nodes into the structural layer, so the graph captures *how* code behaves, not only its syntax — and so a change’s blast radius (dependents of an edited node) is traversable.

15.4 B4. Bug-localisation supervision

Connect bug-fixing commits (and, via SZZ, the bug-inducing commit) to the precise AST nodes they repair, creating explicit “defective-region” labels. This upgrades the task from commit-level classification to *node-level* localisation and gives the GNN a much denser supervision signal.

15.5 B5. Temporal / dynamic-graph modelling

Model the evolving KG natively as a temporal graph (e.g. TGN-style memory over the `author_ts` stream) so the online arrival protocol is *built into* the model rather than approximated by expanding-window retraining — a principled answer to the concept drift documented in §11.

15.6 B6. Evaluation, calibration, and generalisation

Standardise every method on the controlled chronological split (`compare_baselines.py`); add effort-aware metrics (P_{opt} , ACC@20%LOC) and probability calibration (so wvRN’s strong rank-

ing becomes a usable hard decision); and test cross-project transfer by applying the whole pipeline to other ApacheJIT projects. Finally, remove the deprecated [ASTDiff](#)/[ASTEdit](#) layer (§16).

Priority order. For V3 the critical path is **B1 (text graph layer)** → **B2 (heterogeneous GNN)**, evaluated on the controlled split with effort-aware metrics (B6); N1 (rename replay) and N4 (full validation) from Roadmap A should land first so the V3 graph is clean before the GNN consumes it.

16 Legacy Layer

An earlier experiment stored each diff as an [ASTDiff](#) summary node with child [ASTEdit](#) action nodes (COMMIT→HAS_AST_DIFF→ASTDIFF→EDIT→ASTEDIT). It is kept for reference (10 diffs, 491 edits) but superseded by the delta layer, which connects commits to the real AST nodes rather than to standalone summary nodes, enabling traversal into actual code context.

A Example Cypher

Listing 2: A commit’s delta graph (Graph view).

```
MATCH p = (c:Commit)-[r:ADDS|REMOVES|UPDATES|MOVES]->(a:ASTNode)
WHERE c.id STARTS WITH '6ba7af92'
RETURN p;
```

Listing 3: Online evaluation window: history known when a commit arrives.

```
MATCH (cur:Commit {id:$id})
MATCH (past:Commit {in_jit:true}) WHERE past.author_ts < cur.author_ts
RETURN count(past) AS known_history,
sum(CASE WHEN past.buggy THEN 1 ELSE 0 END) AS known_buggy;
```

Listing 4: Average AST-delta size: buggy vs. benign.

```
MATCH (c:Commit {in_jit:true})
OPTIONAL MATCH (c)-[r:ADDS|REMOVES|UPDATES|MOVES]->()
WITH c, count(r) AS delta
RETURN c.buggy AS buggy, count(c) AS commits, round(avg(delta),1) AS
avg_delta;
```

Visual inspection. The script `viz_commit_step.py` produces, for an example commit, the base graph, the file’s AST before the commit, the commit’s delta graph, and a single *overlay* that shows the new nodes/edges (green), removals (red), and updates (orange) laid on the previous structure — the recommended way to visually verify the commit-level modelling.